7

Arrays

OBJETIVOS

Neste capítulo você aprenderá:

- O que são arrays.
- Como utilizar arrays para armazenar dados e recuperá-los de listas e tabelas de valores.
- Como declarar um array, inicializar um array e referenciar elementos individuais de um array.
- Como utilizar a instrução `for` aprimorada para iterar por arrays.
- Como passar arrays para métodos.
- Como declarar e manipular arrays multidimensionais.
- Como escrever métodos que utilizam listas de argumentos de comprimento variável.
- Como ler argumentos da linha de comando em um programa.

7.1 Introdução

7.2 Arrays

7.3 Declarando e criando arrays

7.4 Exemplos que utilizam arrays

7.5 Estudo de caso: simulação de embaralhamento e distribuição de cartas

7.6 A instrução `for` aprimorada

7.7 Passando arrays para métodos

7.8 Estudo de caso: classe `GradeBook` utilizando um array para armazenar notas

7.9 Arrays multidimensionais

7.10 Estudo de caso: classe `GradeBook` utilizando um array bidimensional

7.11 Listas de argumentos de comprimento variável

7.12 Utilizando argumentos de linha de comando

7.13 (Optional) GUI and Graphics Case Study: Drawing Arcs

7.14 (Optional) Software Engineering Case Study: Collaboration Among Objects

7.15 Wrap-Up

7.1 Introdução

- **Arrays**
  - **Estruturas de dados**
  - **Itens de dados de mesmo tipo relacionados**
  - **Mantém o mesmo comprimento uma vez criados**
    - **Entidades de largura fixa**

7.2 Arrays

- **Array**
  - **Grupo de variáveis**
    - **Possuem mesmo tipo**
  - **Tipo por referência**

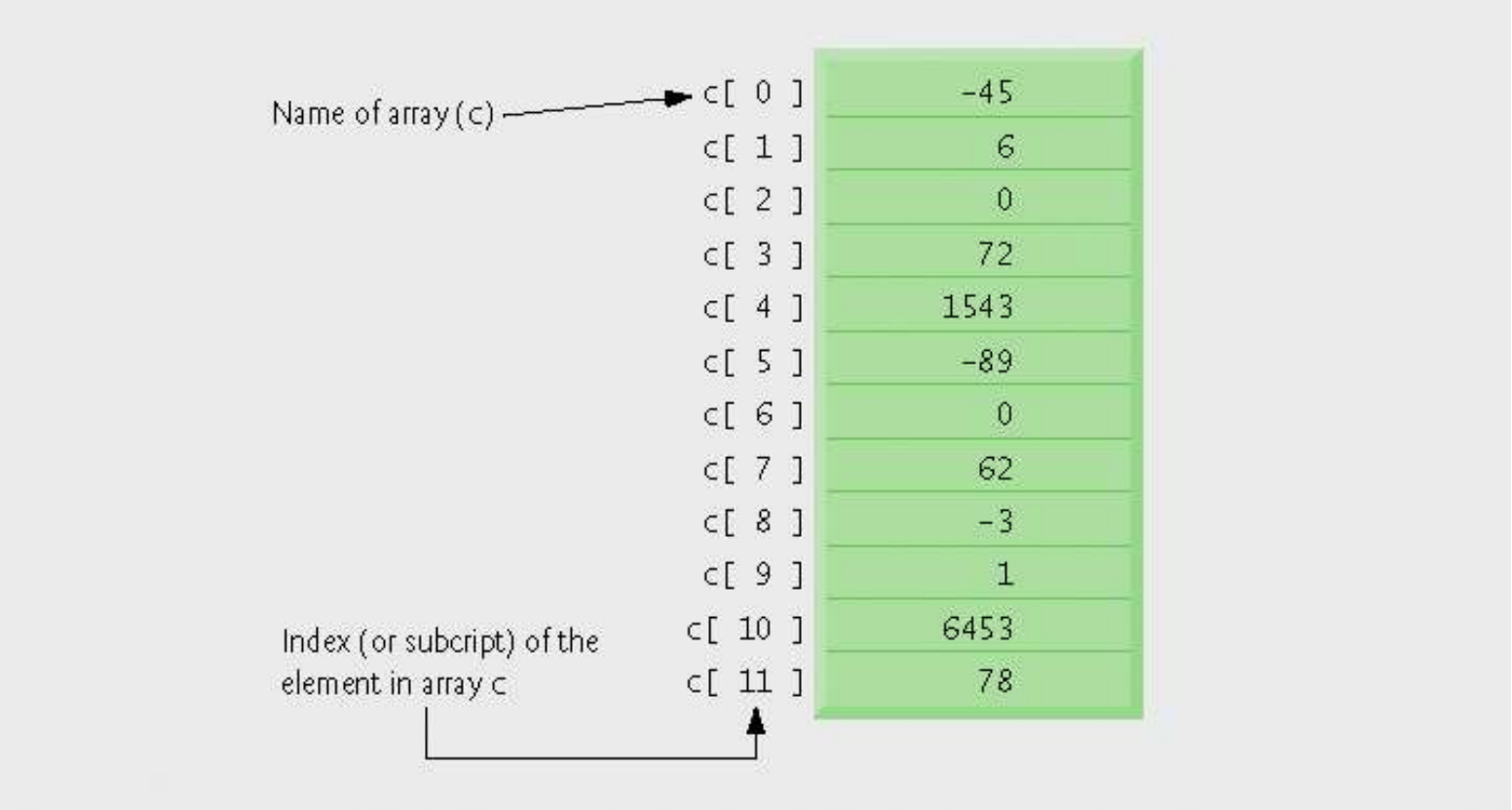


Fig. 7.1 | Um array de 12 elementos.

7.2 Arrays (Cont.)

- **Índice (Index)**
  - **Também chamado de subscrito**
  - **Número de posição entre colchetes**
  - **Deve ser um inteiro não-negativo ou uma expressão inteira**
  - **Primeiro elemento possui índice zero**

```
a = 5;
b = 6;
c[ a + b ] += 2;
```

- **Adiciona** `2` **ao elemento do array** `c[ 11 ]`

Erro comum de programação 7.1

Utilizar um valor de tipo `long` como um índice de array resulta em erro de compilação. Um índice deve ser um valor `int` ou um valor de um tipo que possa ser promovido para int—a saber, `byte`, `short` ou `char`, mas não `long`.

7.2 Arrays (Cont.)

- **Examine o array `c`**
  - **`c` é o *nome* do array**
  - **`c.length` acessa o *comprimento* do array `c`**
  - **`c` possui 12 *elementos* (`c[0]`, `c[1]`, … `c[11]`)**
    - **O *valor* de `c[0]` é `–45`**

7.3 Declarando e criando arrays

- **Declarando e criando arrays**
  - **Arrays são objetos que ocupam espaço na memória**
  - **Criados com a palavra-chave `new`**

```
int c[] = new int[ 12 ];
```

    - **Equivalente a**

```
int c[];  // declara a variável array
c = new int[ 12 ]; // cria o array;
// e atribui à variável array
```

  - **Podemos criar arrays de objetos também**

```
String b[] = new String[ 100 ];
```

Erro comum de programação 7.2

Em uma declaração de array, especificar o número de elementos entre os colchetes da declaração (por exemplo, `int c[ 12 ];`) é um erro de sintaxe.

Boa prática de programação 7.1

Para legibilidade, declare apenas uma variável por declaração. Mantenha cada declaração em uma linha separada e inclua um comentário que descreva a variável sendo declarada.

Erro comum de programação 7.3

Declarar múltiplas variáveis array em uma única declaração pode levar a erros sutis. Considere a declaração `int[] a, b, c;`. Se `a`, `b` e `c` devem ser declarados como variáveis array, então essa declaração é correta—colocar os colchetes logo depois do tipo indica que todos os identificadores na declaração são vairáveis array. Entretanto, se apenas `a` se destina a ser uma variável array, e `b` e `c` variáveis int, então essa declaração é incorreta—a declaração `int a[], b, c;` alcançaria o resultado desejado.

7.4 Exemplos que utilizam arrays

- **Declarando arrays**
- **Criando arrays**
- **Inicializando arrays**
- **Manipulando elementos de arrays**

7.4 Exemplos que utilizam arrays

- **Criando e inicializando array**
  - **Declarar o array**
  - **Criar o array**
  - **Inicializar os elementos do array**

```java
// Fig. 7.2: InitArray.java
// Criando um array.

public class InitArray
{
   public static void main( String args[] )
   {
      int array[]; // declara o array com nome array

      array = new int[ 10 ]; // cria o espaço para o array

      System.out.printf( "%s%8s\n", "Indice", "Valor" ); // tit. colunas

      // gera saída do valor de cada elemnto do array
      for ( int counter = 0; counter < array.length; counter++ )
         System.out.printf( "%5d%8d\n", counter, array[ counter ] );
   } // fim do main
} // fim da classe InitArray
```

```
Indice   Valor
    0       0
    1       0
    2       0
    3       0
    4       0
    5       0
    6       0
    7       0
    8       0
    9       0
```

7.4 Exemplos que utilizam arrays (Cont.)

- **Utilizar um inicializador de array**
  - **Utilize *lista de inicializador***
    - **Ítens envoltos entre chaves (`{}`)**
    - **Ítens em uma lista separada por vírgulas**

```
int n[] = { 10, 20, 30, 40, 50 };
```

      - **Cria um array de cinco elementos**
      - **Valores dos índices `0`, `1`, `2`, `3`, `4`**
  - **Não necessita da palavra-chave `new`**

```java
// Fig. 7.3: InitArray.java
// Inicializando elementos do array com inicializador de array.

public class InitArray
{
   public static void main( String args[] )
   {
      // lista de inicializadores com o valor de cada elemento
      int array[] = { 32, 27, 64, 18, 95, 14, 90, 70, 60, 37 };

      System.out.printf( "%s%8s\n", "Indice", "Valor" ); // tit. Coluna

      // gera a saída do valor de cada elemento do array
      for ( int counter = 0; counter < array.length; counter++ )
         System.out.printf( "%5d%8d\n", counter, array[ counter ] );
   } // fim do main
} // fim da classe InitArray
```

```
Indice   Valor
    0      32
    1      27
    2      64
    3      18
    4      95
    5      14
    6      90
    7      70
    8      60
    9      37
```

7.4 Exemplos que utilizam arrays (Cont.)

- **Calculando o valor para armazenar em cada elemento do array**
  - **Inicializar elementos de uma array de 10 elementos inteiros**

```java
// Fig. 7.4: InitArray.java
// Calculando valores a serem colocados em elementos de um array.

public class InitArray
{
   public static void main( String args[] )
   {
      final int ARRAY_LENGTH = 10; // declara a constante
      int array[] = new int[ ARRAY_LENGTH ]; // create array

      // calcula o valor de cada elemento do array
      for ( int counter = 0; counter < array.length; counter++ )
         array[ counter ] = 2 + 2 * counter;

      System.out.printf( "%s%8s\n", "Indice", "Valor" ); // tit. coluna

      // saída do valor de cada elemnto do array
      for ( int counter = 0; counter < array.length; counter++ )
         System.out.printf( "%5d%8d\n", counter, array[ counter ] );
   } // fim do main
} // fim da classe InitArray
```

```
Indice   Valor
    0       2
    1       4
    2       6
    3       8
    4      10
    5      12
    6      14
    7      16
    8      18
    9      20
```

Boa prática de programação 7.2

Variáveis constantes também são chamadas constantes identificadas ou variáveis de leitura (read-only). Freqüentemente, essas variáveis tornam os programas mais legíveis que os programas que utilizam valores literais (por exemplo, 10)—uma constante identificada como `ARRAY_LENGTH` indica claramente seu propósito, enquanto um valor literal poderia ter diferentes significados com base no contexto em que ele é utilizado.

Erro comum de programação 7.4

Atribuir um valor a uma constante depois de a variável ter sido inicializada é um erro de compilação.

Erro comum de programação 7.5

Tentar utilizar uma constante antes de ela ser inicializada é um erro de compilação.

7.4 Exemplos que utilizam arrays (Cont.)

- **Somando os elementos de um array**
  - **Elementos de array podem representar uma série de valores**
    - **Podemos somar esses valores**

```java
// Fig. 7.5: SumArray.java
// Calculando a soma dos elementos de um array.

public class SumArray
{
   public static void main( String args[] )
   {
      int array[] = { 87, 68, 94, 100, 83, 78, 85, 91, 76, 87 };
      int total = 0;

      // adiciona o valor de cada elemento ao total
      for ( int counter = 0; counter < array.length; counter++ )
         total += array[ counter ];

      System.out.printf( "Soma dos elementos é: %d\n",   total );
   } // fim do main
} // fim da classe SumArray
```

```
Soma dos elementos é: 849
```

7.4 Exemplos que utilizam arrays (Cont.)

- **Utilizando gráficos de barras para exibir dados de array graficamente**
  - **Apresentar dados de forma gráfica**
    - **Por exemplo, gráfico de barras**
  - **Examine a distribuição dos valores**

```java
// Fig. 7.6: BarChart.java
// Impressão em gráfico de barras.

public class BarChart
{
   public static void main( String args[] )
   {
      int array[] = { 0, 0, 0, 0, 0, 0, 1, 2, 4, 2, 1 };

      System.out.println( "Distribuição de valores:" );

      // p/ cada elemento de array, gera a saída de uma barra do gráfico
      for ( int counter = 0; counter < array.length; counter++ )
      {
         //saída de rótulo de barra( "00-09: ", ..., "90-99: ", "100: " )
         if ( counter == 10 )
            System.out.printf( "%5d: ", 100 );
         else
            System.out.printf( "%02d-%02d: ",
               counter * 10, counter * 10 + 9  );

         // imprime a barra de asteriscos
         for ( int stars = 0; stars < array[ counter ]; stars++ )
            System.out.print( "*" );

         System.out.println(); // inicia uma nova linha de saída
      } // fim do for externo
   } // fim do main
} // fim da classe BarChart
```

```
Distribuição de valores:
00-09:
10-19:
20-29:
30-39:
40-49:
50-59:
60-69: *
70-79: **
80-89: ****
90-99: **
  100: *
```

7.4 Exemplos que utilizam arrays (Cont.)

- **Utilizando os elementos de um array como contadores**
  - **Utiliza uma série de contadores para resumir dados**

```java
// Fig. 7.7: RollDie.java
// Rola um dado de seis lados 6000 vezes.
import java.util.Random;

public class RollDie
{
   public static void main( String args[] )
   {
      Random randomNumbers = new Random(); // gerador de número aleatório
      int frequency[] = new int[ 7 ]; // array de cont. de freqüência

      //lança os dados 6000 vezes; usa valor de dado como índice de freq.
      for ( int roll = 1; roll <= 6000; roll++ )
         ++frequency[ 1 + randomNumbers.nextInt( 6 ) ];

      System.out.printf( "%s%10s\n", "Face", "Freqüência" );

      // saída do valor de cada elemento do array
      for ( int face = 1; face < frequency.length; face++ )
         System.out.printf( "%4d%10d\n", face, frequency[ face ] );
   } // fim do main
} // fim da classe RollDie
```

```
Face Freqüência
   1       988
   2       963
   3      1018
   4      1041
   5       978
   6      1012
```

7.4 Exexmplos que utilizam arrays (Cont.)

- **Utilizando arrays para analisar resultados de pesquisas**
  - **40 estudantes odeia avaliam a qualidade da comida**
    - `1-10` **Escala de avaliação:** `1` **significa péssimo,** `10` **significa excelente**
  - **Coloque as 40 respostas em um array de inteiros**
  - **Resuma os resultados da enquete**

```java
// Fig. 7.8: StudentPoll.java
// Programa de análise de enquete.

public class StudentPoll
{
   public static void main( String args[] )
   {
      // array de respostas da pesquisa
      int responses[] = { 1, 2, 6, 4, 8, 5, 9, 7, 8, 10, 1, 6, 3, 8, 6,
         10, 3, 8, 2, 7, 6, 5, 7, 6, 8, 6, 7, 5, 6, 6, 5, 6, 7, 5, 6,
         4, 8, 6, 8, 10 };
      int frequency[] = new int[ 11 ]; // array de cont. freqüência

      // p/ cada resposta seleciona o elemento de respostas e usa o valor
      // como índice de frequência p/ determinar o elemento a incrementar
      for ( int answer = 0; answer < responses.length; answer++ )
         ++frequency[ responses[ answer ] ];

      System.out.printf( "%s%10s", "Avaliação", "Freqüência" );

      // saída do valor de cada elemento do array
      for ( int rating = 1; rating < frequency.length; rating++ )
         System.out.printf( "\n%d%10d", rating, frequency[ rating ] );
   } // fim do main
} // fim da classe StudentPoll
```

Avaliação	Freqüência
1	2
2	2
3	2
4	2
5	5
6	11
7	5
8	7
9	1
10	3

Dica de prevenção de erros 7.1

Uma exceção indica a ocorrência de um erro em um programa. Um programador costuma escrever código para recuperar-se de uma exceção e continuar a execução do programa em vez de terminar o programa anormalmente. Quando um programa tentar acessar um elemento fora dos limites do array, ocorrerá uma `ArrayIndexOutOfBoundsException`. O tratamento de exceções é discutido no capítulo 13.

Dica de prevenção de erros 7.2

Ao escrever código para percorrer um array (fazer laço por um array), assegure-se que o índice de array é sempre maior que ou igual a 0 e menor que o comprimento do array. A condição de continuação do laço deve impedir o acesso a elementos for a desse intervalo.

7.5 Estudo de caso: simulação de embaralhamento e distribuição de cartas

- **Programa que simula embaralhamento e distribuição de cartas**
  - **Utiliza geração de números aleatórios**
  - **Utiliza um array de elementos de tipo por referência para representar as cartas**
  - **Três classes**
    - **Card**
      - **Representa uma carta do baralho que tem uma face e um naipe**
    - **DeckOfCards**
      - **Representa um baralho de 52 cartas**
    - **DeckOfCardsTest**
      - **Demonstra as capacidades de embaralhamento e distribuição de cartas**

```java
// Fig. 7.9: Card.java
// Classe Card representa uma carta de baralho.

public class Card
{
   private String face; // face da carta ("Ace", "Deuce", ...)
   private String suit; // naipe da carta ("Hearts", "Diamonds", ...)

   // construtor de 2 argumentos inicializa face e naipe da carta
   public Card( String cardFace, String cardSuit )
   {
      face = cardFace; // inicializa face da carta
      suit = cardSuit; // inicializa naipe da carta
   } // fim do construtor Card de 2 argumentos

   // retorna a representação String de Card
   public String toString()
   {
      return face + " of " + suit;
   } // fim do método toString
} // fim da classe Card
```

```java
// Fig. 7.10: DeckOfCards.java
// Classe DeckOfCards representa um baralho.
import java.util.Random;

public class DeckOfCards
{
   private Card deck[]; // array de objetos Card
   private int currentCard; // índice do próximo Card a ser distribuído
   private final int NUMBER_OF_CARDS = 52; // número constante de Cards
   private Random randomNumbers; // gerador de número aleatório

   // construtor preenche baralho de cartas
   public DeckOfCards()
   {
      String faces[] = { "Ace", "Deuce", "Three", "Four", "Five", "Six",
         "Seven", "Eight", "Nine", "Ten", "Jack", "Queen", "King" };
      String suits[] = { "Hearts", "Diamonds", "Clubs", "Spades" };

      deck = new Card[ NUMBER_OF_CARDS ]; // cria array de objetos Card
      currentCard = 0; // configura currentCard; deck[ 0 ]
      randomNumbers = new Random(); // cria gerador de número aleatório

      // preenche baralho com objetos Card
      for ( int count = 0; count < deck.length; count++ )
         deck[ count ] =
            new Card( faces[ count % 13 ], suits[ count / 13 ] );
   } // fim do cosntrutor DeckOfCards
```

```
28
29    // embaralha as cartas com algoritmo de uma passagem
30    public void shuffle()
31    {
32       // após embaralhar, distribuição deve iniciar em deck[ 0 ] de novo
33       currentCard = 0; // reinicializa currentCard
34
35       // p/ cada Card, seleciona outro Card aleatório e os compara
36       for ( int first = 0; first < deck.length; first++ )
37       {
38          // seleciona um número aleatório entre 0 e 51
39          int second =  randomNumbers.nextInt( NUMBER_OF_CARDS );
40
41          // compara Card atual com Card aleatoriamente selecionado
42          Card temp = deck[ first ];
43          deck[ first ] = deck[ second ];
44          deck[ second ] = temp;
45       } // fim do for
46    } // fim do método shuffle
47
48    // distribui um Card
49    public Card dealCard()
50    {
51       // determina se ainda há Cards para distribuir
52       if ( currentCard < deck.length )
53          return deck[ currentCard++ ]; // retorna Card atual no array
54       else
55          return null; // retorna null p/ indicar que todos Cards foram
distribuídos
56    } // fim do método dealCard
57 } // fim da classe DeckOfCards
```

```java
// Fig. 7.11: DeckOfCardsTest.java
// Aplicativo de embaralhar e distribuir cartas.

public class DeckOfCardsTest
{
   // executa o aplicativo
   public static void main( String args[] )
   {
      DeckOfCards myDeckOfCards = new DeckOfCards();
      myDeckOfCards.shuffle(); // põe Cards em ordem aleatória

      // imprime todas 52 cartas na ordem em que são distribuídas
      for ( int i = 0; i < 13; i++ )
      {
         // distribui e imprime 4 cartas
         System.out.printf( "%-20s%-20s%-20s%-20s\n",
            myDeckOfCards.dealCard(), myDeckOfCards.dealCard(),
            myDeckOfCards.dealCard(), myDeckOfCards.dealCard() );
      } // fim do for
   } // fim do main
} // fim da classe DeckOfCardsTest
```

Six of Spades	Eight of Spades	Six of Clubs	Nine of Hearts
Queen of Hearts	Seven of Clubs	Nine of Spades	King of Hearts
Three of Diamonds	Deuce of Clubs	Ace of Hearts	Ten of Spades
Four of Spades	Ace of Clubs	Seven of Diamonds	Four of Hearts
Three of Clubs	Deuce of Hearts	Five of Spades	Jack of Diamonds
King of Clubs	Ten of Hearts	Three of Hearts	Six of Diamonds
Queen of Clubs	Eight of Diamonds	Deuce of Diamonds	Ten of Diamonds
Three of Spades	King of Diamonds	Nine of Clubs	Six of Hearts
Ace of Spades	Four of Diamonds	Seven of Hearts	Eight of Clubs
Deuce of Spades	Eight of Hearts	Five of Hearts	Queen of Spades
Jack of Hearts	Seven of Spades	Four of Clubs	Nine of Diamonds
Ace of Diamonds	Queen of Diamonds	Five of Clubs	King of Spades
Five of Diamonds	Ten of Clubs	Jack of Spades	Jack of Clubs

7.6 A estrutura `for` aprimorada

- **Estrutura `for` aprimorada**
  - **Novo recurso do J2SE 5.0**
  - **Permite iterar pelos elementos de um array ou uma coleção sem utilizar um contador**
  - **Sintaxe**

```
for ( parâmetro : nomeDoArray )
    instrução
```

```java
// Fig. 7.12: EnhancedForTest.java
// Utilizando instrução for aprimorada para somar inteiros em um array.

public class EnhancedForTest
{
   public static void main( String args[] )
   {
      int array[] = { 87, 68, 94, 100, 83, 78, 85, 91, 76, 87 };
      int total = 0;

      // adiciona o valor de cada elemento ao total
      for ( int number : array )
         total += number;

      System.out.printf( "Soma dos elementos do array: %d\n", total );
   } // fim do main
} // fim da classe EnhancedForTest
```

```
Soma dos elementos do array: 849
```

7.6 A estrutura `for` aprimorada (Cont.)

- **Linhas 12-13 equivalem a**

```
for ( int counter = 0; counter < array.length; counter++ )
    total += array[ counter ];
```

- **Utilização**
  - **Pode acessar elementos do array**
  - **Não pode modificar elementos do array**
  - **Não pode acessar o contador que indica o número do índice do elemento do array**

7.7 Passando arrays para métodos

- **Como passar array como argumento para um método**
  - **Especifique o nome do array sem nenhum colchete**
    - **Array `hourlyTemperatures` é declarado como**

```
int hourlyTemperatures = new int[ 24 ];
```

    - **A chamada de método**

```
modifyArray( hourlyTemperatures );
```

    - **Passa a referência do array `hourlyTemperatures` para o método `modifyArray`**

```java
// Fig. 7.13: PassArray.java
// Passando arrays e elementos de arrays individuais aos métodos.

public class PassArray
{
   // main cria array e chama modifyArray e modifyElement
   public static void main( String args[] )
   {
      int array[] = { 1, 2, 3, 4, 5 };

      System.out.println(
         "Effects of passing reference to entire array:\n" +
         "The values of the original array are:" );

      // gera saída de elementos do array original
      for ( int value : array )
         System.out.printf( "   %d", value );

      modifyArray( array ); // passa a referência do array
      System.out.println( "\n\nThe values of the modified array are:" );

      // gera saída de elementos do array modificado
      for ( int value : array )
         System.out.printf( "   %d", value );

      System.out.printf(
         "\n\nEffects of passing array element value:\n" +
         "array[3] before modifyElement: %d\n", array[ 3 ] );
```

```java

      modifyElement( array[ 3 ] ); // tenta modificar o array[ 3 ]
      System.out.printf(
         "array[3] after modifyElement: %d\n", array[ 3 ] );
   } // fim do main

   // multiplica cada elemento de um array por 2
   public static void modifyArray( int array2[] )
   {
      for ( int counter = 0; counter < array2.length; counter++ )
         array2[ counter ] *= 2;
   } // fim do método modifyArray

   // multiplica argumento por 2
   public static void modifyElement( int element )
   {
      element *= 2;
      System.out.printf(
         "Value of element in modifyElement: %d\n", element );
   } // fim do método modifyElement
} // fim da classe PassArray
```

```
Effects of passing reference to entire array:
The values of the original array are:
   1   2   3   4   5

The values of the modified array are:
   2   4   6   8   10

Effects of passing array element value:
array[3] before modifyElement: 8
Value of element in modifyElement: 16
array[3] after modifyElement: 8
```

7.7 Passando arrays para métodos (Cont.)

- **Notas sobre a passagem de argumentos para métodos**
  - **Duas maneiras de passar argumentos para métodos**
    - **Passagem por valor (chamada por valor)**
      - **Uma cópia do valor do argumento é passada para o método chamado**
      - **Em Java, todos os argumentos primitivos são passados por valor**
    - **Passagem por referência (chamada por referência)**
      - **O método chamado pode acessar o valor do argumento ho chamador diretamente e modificar os dados**
      - **Método chamado pode manipular esses dados**
      - **Aprimora o desempenho em relação à passagem por valor**
      - **Em Java, todo objeto é passado por referência**
        - **Em Java, arrays são objetos**
        - **Portanto, matrizes (arrays) são passadas aos métodos por referência**

Dica de desempenho 7.1

Passar arrays por referência faz sentido por razões de desempenho. Se arrays fossem passados por valor, uma cópia de cada elemento seria passada. Para arrays grandes e freqüentemente passados, isso desperdiçaria tempo e consumiria considerável capacidade de armazenamento para as cópias dos arrays.

7.8 Estudo de caso: Classe GradeBook utilizando um array para armazenar notas

- **Esta seção desenvolve ainda mais a classe GradeBook**
- **Classe GradeBook**
  - **Representa um livro de notas para armazenar e analisar um conjunto de notas de um estudante**
  - **Não mantém valores individuais das notas**
  - **Repetir os cálculos requer nova digitação das mesmas notas**
    - **Pode ser resolvido armazenando as notas em um array**

```java
// Fig. 7.14: GradeBook.java
// Livro de notas utilizando um array para armazenar notas de teste.

public class GradeBook
{
   private String courseName; // nome do curso p/ esse GradeBook
   private int grades[]; // array de notas de estudante

   // construtor de 2 argumentos inicializa courseName e array de notas
   public GradeBook( String name, int gradesArray[] )
   {
      courseName = name; // inicializa courseName
      grades = gradesArray; // store grades
   } // fim do construtor de 2 argumentos

   // método para configurar o nome do curso
   public void setCourseName( String name )
   {
      courseName = name; // armazena o nome do curso
   } // fim do método setCourseName

   // método para recuperar o nome do curso
   public String getCourseName()
   {
      return courseName;
   } // fim do método getCourseName

```

Atribui a referência do array a variável de instância `grades`

```
28    // exibe uma mensagem de boas-vindas ao usuário GradeBook
29    public void displayMessage()
30    {
31       // getCourseName obtém o nome do curso
32       System.out.printf( "Bem-vindos à disciplina\n%s!\n\n",
33          getCourseName() );
34    } // fim do método displayMessage
35
36    // realiza várias operações nos dados
37    public void processGrades()
38    {
39       // gera a saída no array de notas
40       outputGrades();
41
42       // chama método getAverage para calcular a média
43       System.out.printf( "\nMédia da classe é %.2f\n", getAverage() );
44
45       // chama métodos getMinimum e getMaximum
46       System.out.printf( "Menor nota é %d\nMaior nota é %d\n\n",
47          getMinimum(), getMaximum() );
48
49       // chama outputBarChart p/ imprimir gráfico de distribuição de nota
50       outputBarChart();
51    } // fim do método processGrades
52
53    // localiza nota mínima
54    public int getMinimum()
55    {
56       int lowGrade = grades[ 0 ]; // assume que grades[ 0 ] é menor nota
57
```

```
58         // faz um loop pelo array de notas
59         for ( int grade : grades )
60         {
61            // se nota mais baixa que lowGrade, atribui essa nota a lowGrade
62            if ( grade < lowGrade )
63               lowGrade = grade; // nova nota mais baixa
64         } // fim do for
65
66         return lowGrade; // retorna nota mais baixa
67      } // fim do método getMinimum
68
69      // localiza nota máxima
70      public int getMaximum()
71      {
72         int highGrade = grades[ 0 ]; // assume que grades[ 0 ] maior nota
73
74         // faz um loop pelo array de notas
75         for ( int grade : grades )
76         {
77            // se nota maior que highGrade, atribua essa nota a highGrade
78            if ( grade > highGrade )
79               highGrade = grade; // nova nota mais alta
80         } // fim do for
81
82         return highGrade; // retorna nota mais alta
83      } // fim do método getMaximum
84
```

```java
   // determina média para o teste
   public double getAverage()
   {
      int total = 0; // inicializa o total

      // soma notas de um estudante
      for ( int grade : grades )
         total += grade;

      // retorna média das notas
      return (double) total / grades.length;
   } // fim do metodo getAverage

   // gera saída do gráfico de barras com distribuição de notas
   public void outputBarChart()
     {
        System.out.println( "Distribuição de notas:" );

        // armazena frequência de notas em cada intervalo de 10 notas
        int frequency[] = new int[ 11 ];

        // p/ cada nota incrementa a frequência apropriada
        for ( int grade : grades )
           ++frequency[ grade / 10 ];

```

```
110         // p/ cada frequência de nota imprime barra no gráfico
111         for ( int count = 0; count < frequency.length; count++ )
112         {
113            // rótulo de barra ( "00-09: ", ..., "90-99: ", "100: " )
114            if ( count == 10 )
115               System.out.printf( "%5d: ", 100 );
116            else
117               System.out.printf( "%02d-%02d: ",
118                  count * 10, count * 10 + 9  );
119
120            // imprime a barra de asteriscos
121            for ( int stars = 0; stars < frequency[ count ]; stars++ )
122               System.out.print( "*" );
123
124            System.out.println(); // pula uma linha
125         } // fim do for externo
126      } // fim do método outputBarChart
127
128      // saída do conteúdo do array de notas
129      public void outputGrades()
130      {
131         System.out.println( "As notas são:\n" );
132
133         // saída da nota de cada estudante
134         for ( int student = 0; student < grades.length; student++ )
135            System.out.printf( "Estudante %2d: %3d\n",
136               student + 1, grades[ student ] );
137      } // fim do método outputGrades
138   } // fim da classe GradeBook
```

Observação de engenharia de software 7.1

Um aplicativo de teste é responsável por criar um objeto da classe sendo testado e fornecer-lhe dados. Esses dados poderiam vir de qualquer uma das várias fontes. Os dados de teste podem ser colocados diretamente em um array com um inicializador de array, podem vir do teclado, de um arquivo (como será apresentado no capítulo 14) ou de uma rede (como será apresentado no capítulo 22). Depois de passar esses dados para o construtor da classe para instanciar o objeto, o aplicativo de teste deve chamar o objeto para testar seus métodos e manipular seus dados. Reunir os dados em um aplicativo de teste como esse permite à classe manipular dados de várias fontes.

```java
// Fig. 7.15: GradeBookTest.java
// Cria objeto GradeBook utilizando um array de notas.

public class GradeBookTest
{
   // método main inicia a execução do programa
   public static void main( String args[] )
   {
      // array de notas de estudante
      int gradesArray[] = { 87, 68, 94, 100, 83, 78, 85, 91, 76, 87 };

      GradeBook myGradeBook = new GradeBook(
         "Algoritmos e linguagem de programação II", gradesArray );
      myGradeBook.displayMessage();
      myGradeBook.processGrades();
   } // fim do main
} // fim da classe GradeBookTest
```

```
Bem-vindos à disciplina
Algoritmos e linguagem de programação II!

As notas são:

Estudante  1:  87
Estudante  2:  68
Estudante  3:  94
Estudante  4: 100
Estudante  5:  83
Estudante  6:  78
Estudante  7:  85
Estudante  8:  91
Estudante  9:  76
Estudante 10:  87

Média da classe é 84.90
Menor nota é 68
Maior nota é 100

Distribuição de notas:
00-09:
10-19:
20-29:
30-39:
40-49:
50-59:
60-69: *
70-79: **
80-89: ****
90-99: **
  100: *
```

7.9 Arrays Multidimensionais

- **Arrays com várias dimensões**
  - **Tabelas com linhas e colunas**
    - **Array bidimensional**
    - **Array m-por-n**

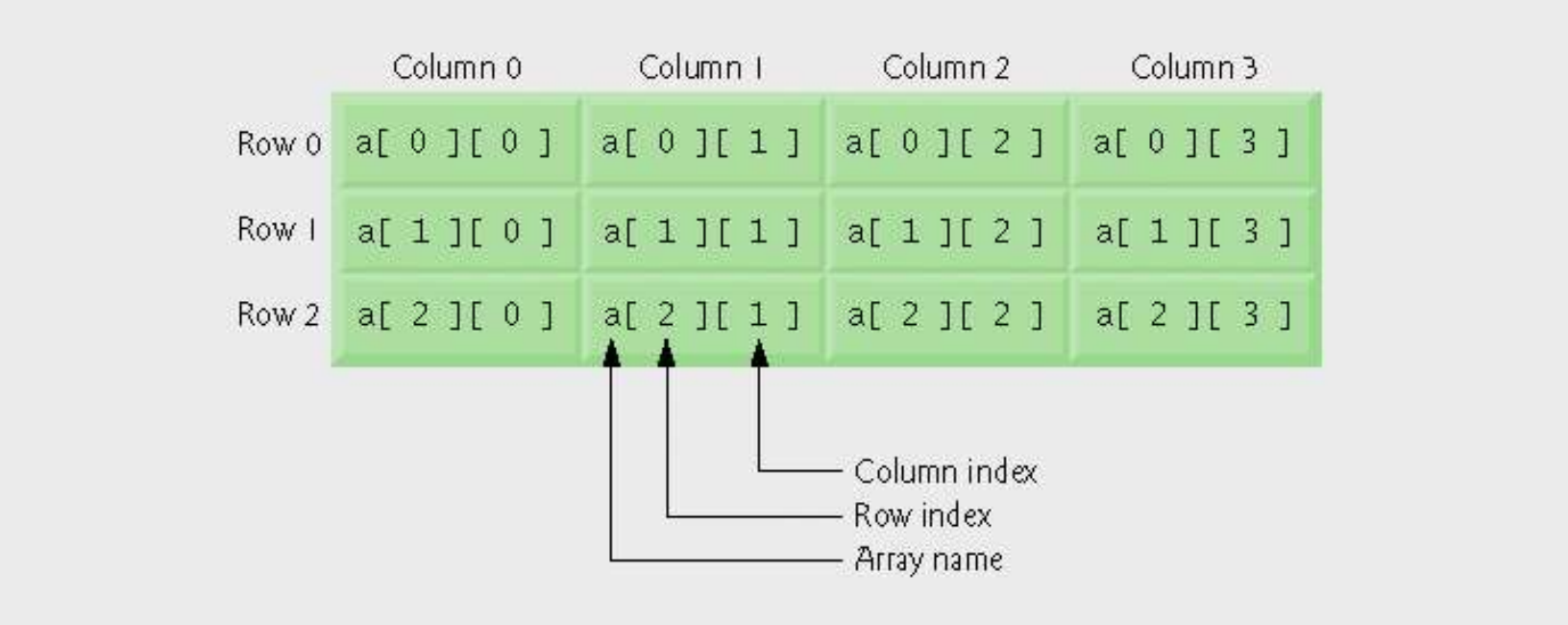


Fig. 7.16 | Array bidimensional com três linhas e quatro colunas.

7.9 Arrays Multidimensionais (Cont.)

- **Arrays de arrays unidimensionais**
  - **Declaração de array com duas dimensões** `b[2][2]`

```
int b[][] = { { 1, 2 }, { 3, 4 } };
```

    - `1` **e** `2` **inicializam** `b[0][0]` **e** `b[0][1]`
    - `3` **e** `4` **inicializam** `b[1][0]` **e** `b[1][1]`

```
int b[][] = { { 1, 2 }, { 3, 4, 5 } };
```

    - **linha** `0` **contém elementos** `1` **e** `2`
    - **linha** `1` **contém elementos** `3`, `4` **e** `5`

7.9 Arrays Multidimensionais (Cont.)

- **Arrays bidimensionais com linhas de diferentes comprimentos**
  - **Comprimentos das linhas em um array não precisam ser os mesmos**
    - **Por exemplo,**

```
int b[][] = { { 1, 2 }, { 3, 4, 5 } };
```

7.9 Arrays Multidimensionais (Cont.)

- **Criando arrays bidimensionais com expressões de criação de arrays**
  - **Podem ser criados dinamicamente**
    - **Array 3-por-4**

```
int b[][];
b = new int[ 3 ][ 4 ];
```

    - **Linhas podem ter diferenets números de colunas**

```
int b[][];
b = new int[ 2 ][ ];    // cria 2 linhas
b[ 0 ] = new int[ 5 ]; // cria 5 colunas para linha 0
b[ 1 ] = new int[ 3 ]; // cria 3 colunas para linha 1
```

```java
// Fig. 7.17: InitArray.java
// Inicializando arrays bidimensionais.

public class InitArray
{
   // cria e gera saída de arrays bidimensionais
   public static void main( String args[] )
   {
      int array1[][] = { { 1, 2, 3 }, { 4, 5, 6 } };
      int array2[][] = { { 1, 2 }, { 3 }, { 4, 5, 6 } };

      System.out.println( "Valores no array1 por linha são" );
      outputArray( array1 ); // exibe array1 por linha

      System.out.println( "\nValores no array2 por linha são" );
      outputArray( array2 ); // exibe array2 por linha
   } // fim do main

```

```java
   // gera saída de linhas e colunas de um array bidimensional
   public static void outputArray( int array[][] )
   {
      // faz um loop pelas linhas do array
      for ( int row = 0; row < array.length; row++ )
      {
         // faz um loop pelas colunas da linha atual
         for ( int column = 0; column < array[ row ].length; column++ )
            System.out.printf( "%d  ", array[ row ][ column ] );

         System.out.println(); // pula uma linha
      } // fim do for externo
   } // fim do método outputArray
} // fim da classe InitArray
```

```
Valores no array1 por linha são
1  2  3
4  5  6

Valores no array2 por linha são
1  2
3
4  5  6
```

7.9 Arrays Multidimensionais (Cont.)

- **Manipulações comuns de arrays multidimensionais realizadas com a instrução `for`**
  - **Muitas manipulações de array comuns utilizam a instrução `for`**

Por exemplo,

```
for ( int column = 0; column < a[ 2 ].length; column++ )
      a[ 2 ][ column ] = 0;
```

7.10 Estudo de caso: Classe `GradeBook` utilizando um array bidimensional

- Classe `GradeBook`
  - Array unidimensional
    - Armazena notas de estudantes de uma única prova
  - Array bidimensional
    - Armazena notas de um estudante e da classe como um todo

```java
// Fig. 7.18: GradeBook.java
// Grade book using a two-dimensional array to store grades.

public class GradeBook
{
   private String courseName; // name of course this grade book

   private int grades[][]; // two-dimensional array of student grades

   // two-argument constructor initializes courseName and grades array
   public GradeBook( String name, int gradesArray[][] )
   {
      courseName = name; // initialize courseName
      grades = gradesArray; // store grades
   } // end two-argument GradeBook constructor

   // method to set the course name
   public void setCourseName( String name )
   {
      courseName = name; // store the course name
   } // end method setCourseName

   // method to retrieve the course name
   public String getCourseName()
   {
      return courseName;
   } // end method getCourseName

```

```
   // display a welcome message to the GradeBook user
   public void displayMessage()
   {
      // getCourseName gets the name of the course
      System.out.printf( "Welcome to the grade book for\n%s!\n\n",
         getCourseName() );
   } // end method displayMessage

   // perform various operations on the data
   public void processGrades()
   {
      // output grades array
      outputGrades();

      // call methods getMinimum and getMaximum
      System.out.printf( "\n%s %d\n%s %d\n\n",
         "Lowest grade in the grade book is", getMinimum(),
         "Highest grade in the grade book is", getMaximum() );

      // output grade distribution chart of all grades on all tests
      outputBarChart();
   } // end method processGrades

   // find minimum grade
   public int getMinimum()
   {
      // assume first element of grades array is smallest
      int lowGrade = grades[ 0 ][ 0 ];

```

```
57        // loop through rows of grades array
58        for ( int studentGrades[] : grades )
59        {
60           // loop through columns of current row
61           for ( int grade : studentGrades )
62           {
63              // if grade less than lowGrade, assign it to lowGrade
64              if ( grade < lowGrade )
65                 lowGrade = grade;
66           } // end inner for
67        } // end outer for
68
69        return lowGrade; // return lowest grade
70     } // end method getMinimum
71
72     // find maximum grade
73     public int getMaximum()
74     {
75        // assume first element of grades array is largest
76        int highGrade = grades[ 0 ][ 0 ];
77
```

```
78        // loop through rows of grades array
79        for ( int studentGrades[] : grades )
80        {
81           // loop through columns of current row
82           for ( int grade : studentGrades )
83           {
84              // if grade greater than highGrade, assign it to highGrade
85              if ( grade > highGrade )
86                 highGrade = grade;
87           } // end inner for
88        } // end outer for
89
90        return highGrade; // return highest grade
91     } // end method getMaximum
92
93     // determine average grade for particular set of grades
94     public double getAverage( int setOfGrades[] )
95     {
96        int total = 0  // initialize total
97
98        // sum grades for one student
99        for ( int grade : setOfGrades )
100          total += grade;
101
102        // return average of grades
103        return (double) total / setOfGrades.length;
104     } // end method getAverage
105
```

```java
    // output bar chart displaying overall grade distribution
    public void outputBarChart()
    {
       System.out.println( "Overall grade distribution:" );

       // stores frequency of grades in each range of 10 grades
       int frequency[] = new int[ 11 ];

       // for each grade in GradeBook, increment the appropriate frequency
       for ( int studentGrades[] : grades )
       {
          for ( int grade : studentGrades )
             ++frequency[ grade / 10 ];
       } // end outer for

       // for each grade frequency, print bar in chart
       for ( int count = 0; count < frequency.length; count++ )
       {
          // output bar label ( "00-09: ", ..., "90-99: ", "100: " )
          if ( count == 10 )
             System.out.printf( "%5d: ", 100 );
          else
             System.out.printf( "%02d-%02d: ",
                count * 10, count * 10 + 9  );

          // print bar of asterisks
          for ( int stars = 0; stars < frequency[ count ]; stars++ )
             System.out.print( "*" );
```

```java

            System.out.println(); // start a new line of output
         } // end outer for
      } // end method outputBarChart

      // output the contents of the grades array
      public void outputGrades()
      {
         System.out.println( "The grades are:\n" );
         System.out.print( "            " ); // align column heads

         // create a column heading for each of the tests
         for ( int test = 0; test < grades[ 0 ].length; test++ )
            System.out.printf( "Test %d  ", test + 1 );

         System.out.println( "Average" ); // student average column heading

         // create rows/columns of text representing array grades
         for ( int student = 0; student < grades.length; student++ )
         {
            System.out.printf( "Student %2d", student + 1 );

            for ( int test : grades[ student ] ) // output student's grades
               System.out.printf( "%8d", test );

```

```java
            // call method getAverage to calculate student's average grade;
            // pass row of grades as the argument to getAverage
            double average = getAverage( grades[ student ] );
            System.out.printf( "%9.2f\n", average );
        } // end outer for
    } // end method outputGrades
} // end class GradeBook
```

```java
// Fig. 7.19: GradeBookTest.java
// Creates GradeBook object using a two-dimensional array of grades.

public class GradeBookTest
{
   // main method begins program execution
   public static void main( String args[] )
   {
      // two-dimensional array of student grades
      int gradesArray[][] = { { 87, 96, 70 },
                              { 68, 87, 90 },
                              { 94, 100, 90 },
                              { 100, 81, 82 },
                              { 83, 65, 85 },
                              { 78, 87, 65 },
                              { 85, 75, 83 },
                              { 91, 94, 100 },
                              { 76, 72, 84 },
                              { 87, 93, 73 } };

      GradeBook myGradeBook = new GradeBook(
         "CS101 Introduction to Java Programming", gradesArray );
      myGradeBook.displayMessage();
      myGradeBook.processGrades();
   } // end main
} // end class GradeBookTest
```

```
Welcome to the grade book for
CS101 Introduction to Java Programming!

The grades are:

            Test 1  Test 2  Test 3  Average
Student  1      87      96      70    84.33
Student  2      68      87      90    81.67
Student  3      94     100      90    94.67
Student  4     100      81      82    87.67
Student  5      83      65      85    77.67
Student  6      78      87      65    76.67
Student  7      85      75      83    81.00
Student  8      91      94     100    95.00
Student  9      76      72      84    77.33
Student 10      87      93      73    84.33

Lowest grade in the grade book is 65
Highest grade in the grade book is 100

Overall grade distribution:
00-09:
10-19:
20-29:
```

```
30-39:
40-49:
50-59:
60-69: ***
70-79: ******
80-89: ***********
90-99: *******
  100: ***
```

7.11 Listas de argumentos de comprimento variável

- **Listas de argumentos de comprimento variável**
  - **Novo recurso do J2SE 5.0**
  - **Número não especificado de argumentos**
  - **Utilize reticências (...) em uma lista de parâmetros de método**
    - **Esse uso de reticências só pode ocorrer uma única vez em uma listas de parâmetros**
    - **As reticências e seu tipo devem ser colocadas no fim da lista de parâmetros**
  - **Array em que seus elementos são todos do mesmo tipo**

```java
// Fig. 7.20: VarargsTest.java
// Utilizando listas de argumentos de comprimento variável.

public class VarargsTest
{
   // calcula a média
   public static double average( double... numbers )
   {
      double total = 0.0; // inicializa variável total

      // calcula total utilizando a instrução for aprimorada
      for ( double d : numbers )
         total += d;

      return total / numbers.length;
   } // fim do método average

   public static void main( String args[] )
   {
      double d1 = 10.0;
      double d2 = 20.0;
      double d3 = 30.0;
      double d4 = 40.0;

```

```java
      System.out.printf( "d1 = %.1f\nd2 = %.1f\nd3 = %.1f\nd4 = %.1f\n\n",
         d1, d2, d3, d4 );

      System.out.printf( "Average of d1 and d2 is %.1f\n",
         average( d1, d2 ) );
      System.out.printf( "Average of d1, d2 and d3 is %.1f\n",
         average( d1, d2, d3 ) );
      System.out.printf( "Average of d1, d2, d3 and d4 is %.1f\n",
         average( d1, d2, d3, d4 ) );
   } // fim do main
} // fim da classe VarargsTest
```

```
d1 = 10.0
d2 = 20.0
d3 = 30.0
d4 = 40.0

Average of d1 and d2 is 15.0
Average of d1, d2 and d3 is 20.0
Average of d1, d2, d3 and d4 is 25.0
```

Erro comum de programação 7.6

Colocar reticências no meio de uma lista de parâmetros de método é um erro sintaxe. As reticências só podem ser colocadas no fim da lista de parâmetros.

7.12 Utilizando argumentos de linha de comando

- **Argumentos de linha de comando**
  - **Passar argumentos a partir da linha de comando**
    - `String args[]`
  - **Aparece depois do nome da classe no comando `java`**
    - `java MyClass a b`
  - **Número de argumentos passados na linha de comando**
    - `args.length`
  - **Primeiro argumento de linha de comando**
    - `args[ 0 ]`

```java
// Fig. 7.21: InitArray.java
// Utilizando argumentos de linha de comando para inicializar um array.

public class InitArray
{
   public static void main( String args[] )
   {
      // verifica número de argumentos de linha de comando
      if ( args.length != 3 )
         System.out.println(
            "Error: Please re-enter the entire command, including\n" +
            "an array size, initial value and increment." );
      else
      {
         // obtém o tam do array a partir do prim argumento de linha de comando
         int arrayLength = Integer.parseInt( args[ 0 ] );
         int array[] = new int[ arrayLength ]; // cria o array

         // obtém o valor inicial e o incremento a partir do argum de linha de comando
         int initialValue = Integer.parseInt( args[ 1 ] );
         int increment = Integer.parseInt( args[ 2 ] );

         // calcula o valor de cada elemento do array
         for ( int counter = 0; counter < array.length; counter++ )
            array[ counter ] = initialValue + increment * counter;

         System.out.printf( "%s%8s\n", "Index", "Value" );

```

```
29          // exibe o valor e o índice do array
30          for ( int counter = 0; counter < array.length; counter++ )
31             System.out.printf( "%5d%8d\n", counter, array[ counter ] );
32       } // fim do else
33    } // fim do main
34 } // fim da classe InitArray
```

```
java InitArray
Error: Please re-enter the entire command, including
an array size, initial value and increment.
```

```
java InitArray 5 0 4
Index   Value
    0       0
    1       4
    2       8
    3      12
    4      16
```

```
java InitArray 10 1 2
Index   Value
    0       1
    1       3
    2       5
    3       7
    4       9
    5      11
    6      13
    7      15
    8      17
    9      19
```

Referências

Adaptado de: “Teaching Resources for Java How to Program, 6/e”.

Deitel & Deitel. Java como programar, 6 ed.

Kathy Sierra e Bert Bates. Head First Java!.

Notas de aula da disciplina Projeto com Orientação a Objetos.

7.13 (Optional) GUI and Graphics Case Study: Drawing Arcs

- **Draw rainbow**
  - **Use arrays**
  - **Use repetition statement**
  - **Use `Graphics` method `fillArc`**

Outline

DrawRainbow.java

(1 of 2)

Line 22

```
1  // Fig. 7.22: DrawRainbow.java
2  // Demonstrates using colors in an array.
3  import java.awt.Color;
4  import java.awt.Graphics;
5  import javax.swing.JPanel;
6
7  public class DrawRainbow extends JPanel
8  {
9     // Define indigo and violet
10    final Color VIOLET = new Color( 128, 0, 128 );
11    final Color INDIGO = new Color( 75, 0, 130 );
12
13    // colors to use in the rainbow, starting from the innermost
14    // The two white entries result in an empty arc in the center
15    private Color colors[] =
16       { Color.WHITE, Color.WHITE, VIOLET, INDIGO, Color.BLUE,
17         Color.GREEN, Color.YELLOW, Color.ORANGE, Color.RED };
18
19    // constructor
20    public DrawRainbow()
21    {
22       setBackground( Color.WHITE ); // set the background to white
23    } // end DrawRainbow constructor
24
25    // draws a rainbow using
26    public void paintComponent( Graphics g )
27    {
28       super.paintComponent( g );
29
30       int radius = 20; // radius of an arch
```

Set component background to white

```
31
32      // draw the rainbow near the bottom-center
33      int centerX = getWidth() / 2;
34      int centerY = getHeight() - 10;
35
36      // draws filled arcs starting with the outermost
37      for ( int counter = colors.length; counter > 0; counter-- )
38      {
39         // set the color for the current arc
40         g.setColor( colors[ counter - 1 ] );
41
42         // fill the arc from 0 to 180 degrees
43         g.fillArc( centerX - counter * radius,
44            centerY - counter * radius,
45            counter * radius * 2, counter * radius * 2, 0, 180 );
46      } // end for
47   } // end method paintComponent
48 } // end class DrawRainbow
```

Draw a filled semicircle

Outline

DrawRainbow

.java

(2 of 2)

Lines 43-45

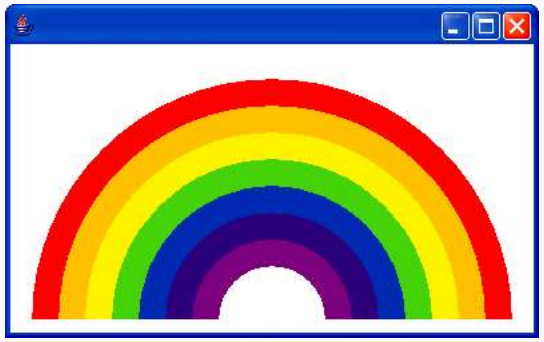
Outline

DrawRainbowTest.java

```java
// Fig. 7.23: DrawRainbowTest.java
// Test application to display a rainbow.
import javax.swing.JFrame;

public class DrawRainbowTest
{
   public static void main( String args[] )
   {
      DrawRainbow panel = new DrawRainbow();
      JFrame application = new JFrame();

      application.setDefaultCloseOperation( JFrame.EXIT_ON_CLOSE );
      application.add( panel );
      application.setSize( 400, 250 );
      application.setVisible( true );
   } // end main
} // end class DrawRainbowTest
```



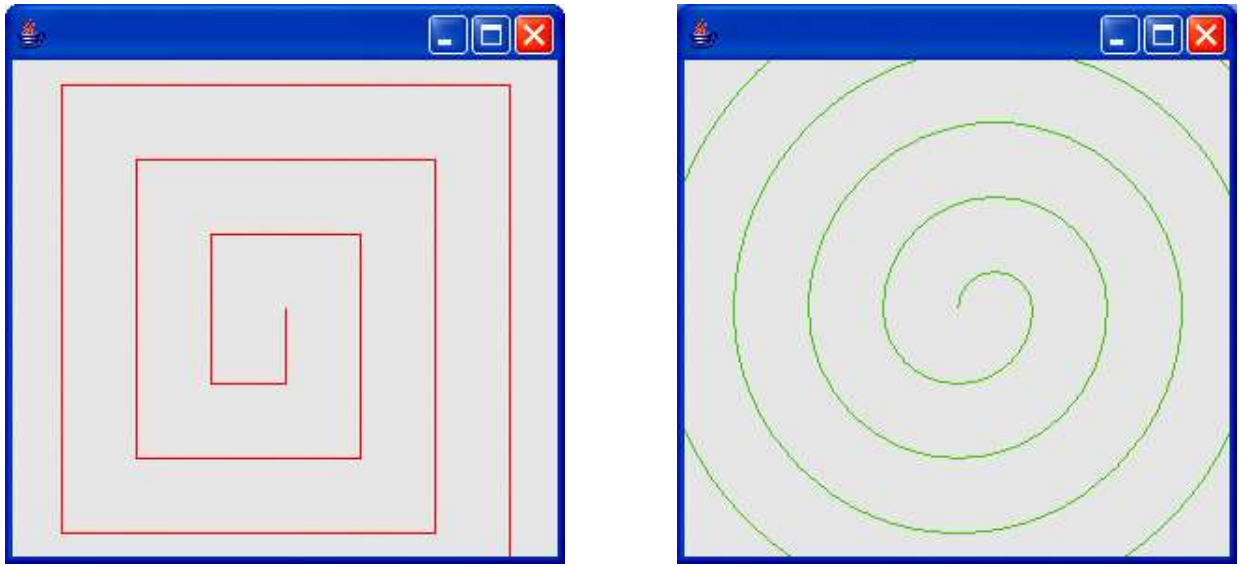


Fig. 7.24 | Drawing a spiral using `drawLine` (left) and `drawArc` (right).

7.14 (Optional) Software Engineering Case Study: Collaboration Among Objects

- **Collaborations**
  - **When objects communicate to accomplish task**
    - **Accomplished by invoking operations (methods)**
  - **One object sends a message to another object**

7.14 (Optional) Software Engineering Case Study (Cont.)

- **Identifying the collaborations in a system**
  - **Read requirements document to find**
    - **What ATM should do to authenticate a use**
    - **What ATM should do to perform transactions**
  - **For each action, decide**
    - **Which objects must interact**
      - **Sending object**
      - **Receiving object**

An object of class…	sends the message…	to an object of class…
ATM	displayMessage	Screen
	getInput	Keypad
	authenticateUser	BankDatabase
	execute	BalanceInquiry
	execute	Withdrawal
	Execute	Deposit
BalanceInquiry	getAvailableBalance	BankDatabase
	getTotalBalance	BankDatabase
	displayMessage	Screen
Withdrawal	displayMessage	Screen
	getInput	Keypad
	getAvailableBalance	BankDatabase
	isSufficientCashAvailable	CashDispenser
	debit	BankDatabase
	dispenseCash	CashDispenser
Deposit	displayMessage	Screen
	getInput	Keypad
	isEnvelopeReceived	DepositSlot
	Credit	BankDatabase
BankDatabase	validatePIN	Account
	getAvailableBalance	Account
	getTotalBalance	Account
	debit	Account
	Credit	Account

Fig. 7.25 | Collaborations in the ATM system.

7.14 (Optional) Software Engineering Case Study (Cont.)

- **Interaction Diagrams**
  - **Model interactions use UML**
  - **Communication diagrams**
    - **Also called collaboration diagrams**
    - **Emphasize which objects participate in collaborations**
  - **Sequence diagrams**
    - **Emphasize when messages are sent between objects**

7.14 (Optional) Software Engineering Case Study (Cont.)

- **Communication diagrams**
  - **Objects**
    - **Modeled as rectangles**
    - **Contain names in the form `objectName:className`**
  - **Objects are connected with solid lines**
  - **Messages are passed alone these lines in the direction shown by arrows**
  - **Name of message appears next to the arrow**

Fig. 7.26 | **Communication diagram of the ATM executing a balance inquiry.**

7.14 (Optional) Software Engineering Case Study (Cont.)

- **Sequence of messages in a communication diagram**
  - **Appear to the left of a message name**
  - **Indicate the order in which the message is passed**
  - **Process in numerical order from least to greatest**

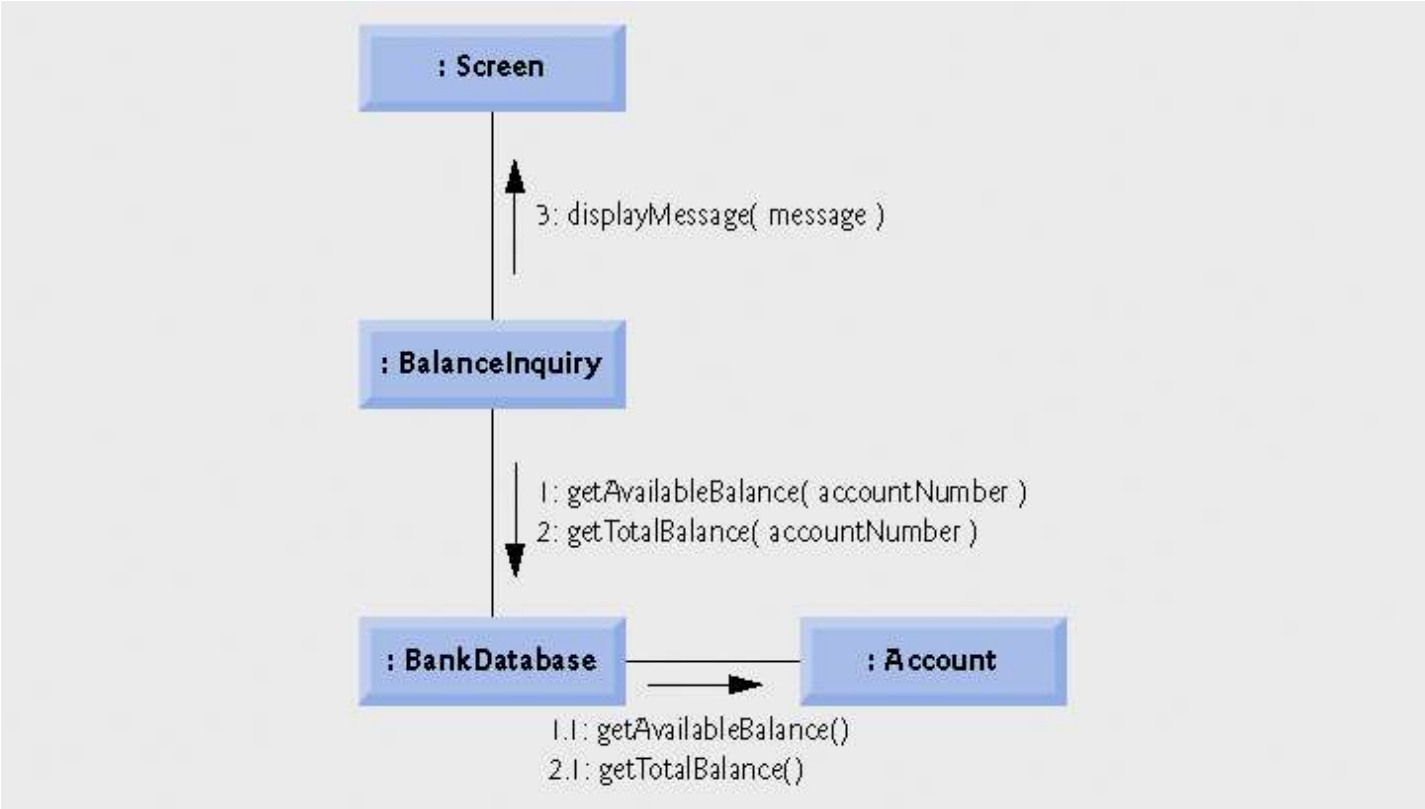


Fig. 7.27 | Communication diagram for executing a balance inquiry.

7.14 (Optional) Software Engineering Case Study (Cont.)

- **Sequence diagrams**
  - **Help model the timing of collaborations**
  - **Lifeline**
    - **Dotted line extending down from an object's rectangle**
      - **Represents the progression of time**
  - **Activation**
    - **Thin vertical rectangle**
      - **Indicates that an object is executing**

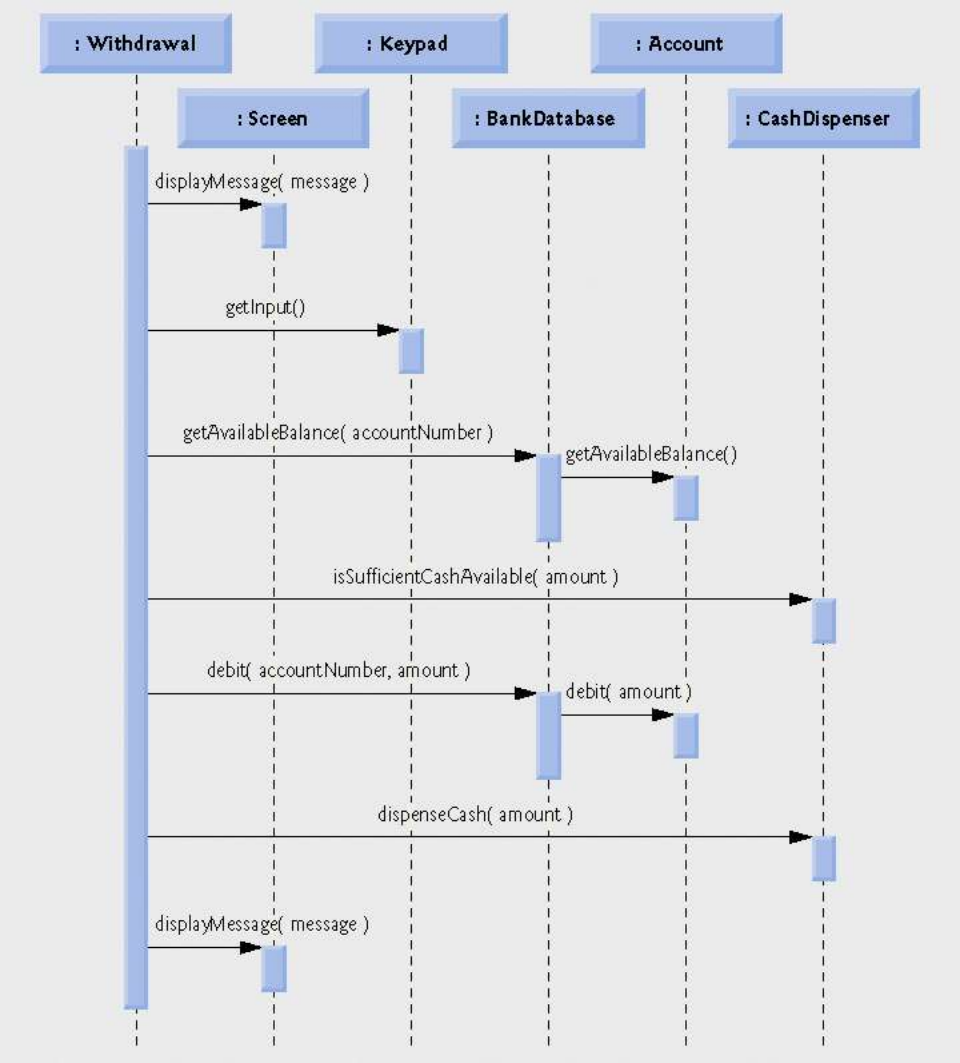


Fig. 7.28 | Sequence diagram that models a `Withdrawal` executing.

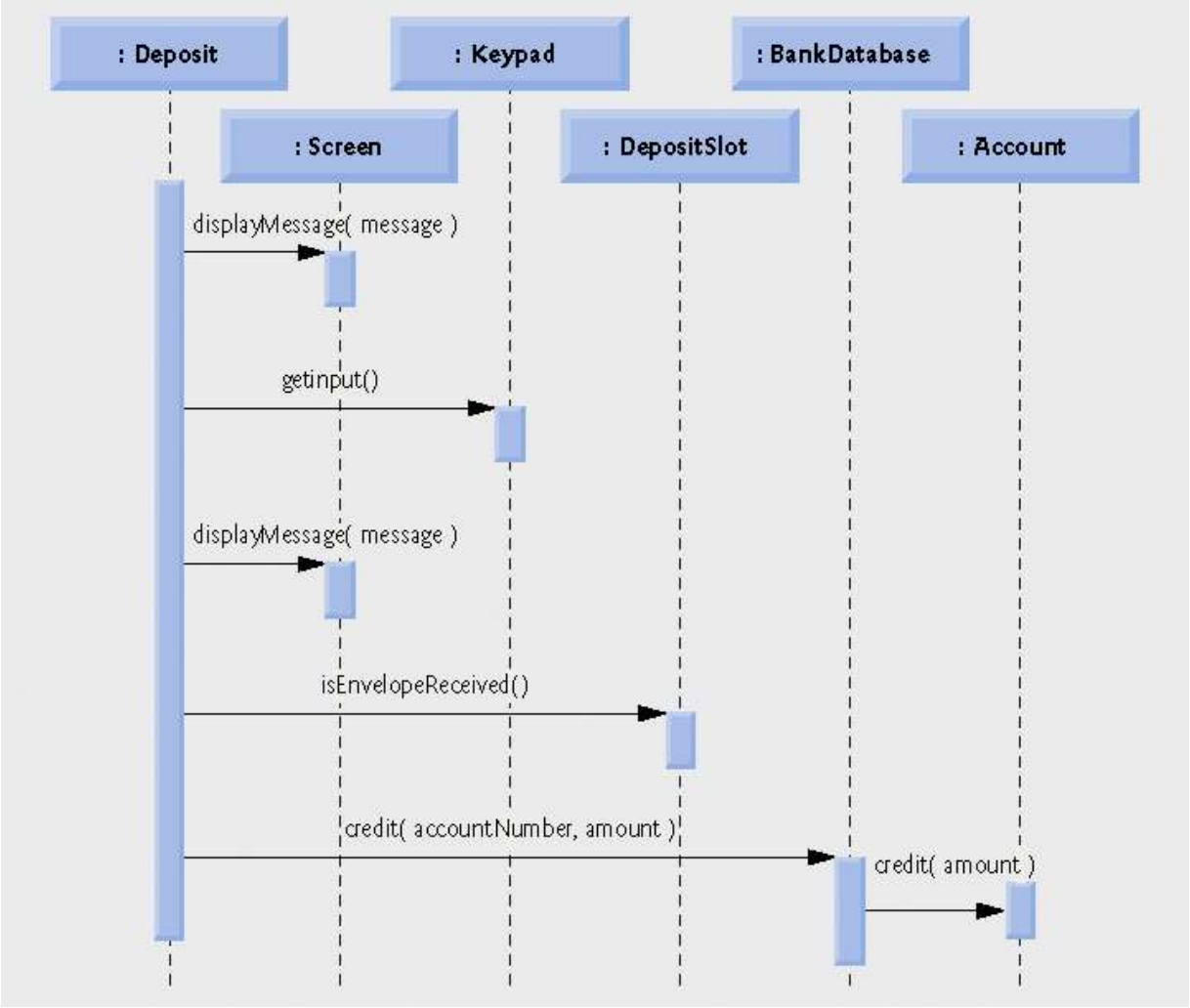


Fig. 7.29 | Sequence diagram that models a `Deposit` executing.